

```
1  /*
2   * OST - Uebungen 'Algorithmen & Datenstrukturen (AlgDat)'
3   * Version: Sun Sep 15 17:57:14 CEST 2024
4   */
5
6  package ex01.solution.task04;
7
8  import java.util.ArrayList;
9  import java.util.Random;
10
11  public class BinarySearchArrayTest {
12
13      protected ArrayList<Integer> arrayList;
14
15      public BinarySearchArrayTest() {
16          arrayList = new ArrayList<>();
17      }
18
19      public void clear() {
20          arrayList = new ArrayList<>();
21      }
22
23      public void generateTree(int nodes) {
24          for (int i: new Random().ints(nodes, 0, Integer.MAX_VALUE).toArray()) {
25              if (arrayList.size() == 0)
26                  arrayList.add(i);
27              else
28                  add(0, arrayList.size() - 1, i);
29          }
30      }
31
32      /**
33       * Adds 'content' recursively into the ArrayList by applying a Binary-Search.
34       *
35       * @param lower The lower bound (inclusive) of the range where to insert the content
36       *
37       * @param upper The upper bound (inclusive) of the range where to insert the content
38       *
39       * @param content The number to insert into the ArrayList.
40       */
41      public void add(int lower, int upper, int content) {
42          if (lower == upper) { // we found the insert-position
43              if (content >= arrayList.get(lower)) {
44                  arrayList.add(lower+1, content);
45              } else {
46                  arrayList.add(lower, content);
47              }
48              return;
49          }
50          // we have to search further:
51          int middle = (lower + upper) / 2;
52          if (content > arrayList.get(middle)) {
53              add(middle+1, upper, content);
54          } else {
55              add(lower, middle, content);
56          }
57      }
58
59      public boolean verify(int size, boolean exiting) {
60          int arrayListSize = arrayList.size();
61          if (arrayListSize != size) {
62              System.err.println("ERROR: bad size: " + arrayListSize);
63              if (exiting) {
64                  System.exit(1);
65              } else {
66                  return false;
67              }
68          }
69      }
70  }
```

15.9.2024 17:57:14

BinarySearchArrayTest.java

Page 2/2

```

67     int lhs = Integer.MIN_VALUE;
68     boolean failure = false;
69     for (int i = 0; i < arrayList.size(); i++) {
70         int rhs = arrayList.get(i);
71         if (lhs > rhs) {
72             System.out.format("ERROR: wrong order at [%d]: %d > %d\n", i, lhs, rhs);
73             failure = true;
74             break;
75         }
76         lhs = rhs;
77     }
78     if (failure) {
79         if (arrayListSize < 20) {
80             System.out.println(arrayList);
81         }
82         if (exiting) {
83             System.exit(2);
84         } else {
85             return false;
86         }
87     }
88     return true;
89 }
90
91 public static void main(String[] args) {
92     System.out.println("ARRAYLIST based TEST");
93     System.out.println("Please be patient, the following operations may take some time
...");
94     final int BEGINSIZE = 10000;
95     final int TESTRUNS = 100;
96     final int VARYSIZE = 10;
97
98     BinarySearchArrayTest binarySearchArray = new BinarySearchArrayTest();
99     double avgTime = 0;
100    long startTime;
101    for (int i = 0; i < TESTRUNS; i++) {
102        binarySearchArray.clear();
103        startTime = System.currentTimeMillis();
104        int size = BEGINSIZE + i * VARYSIZE;
105        binarySearchArray.generateTree(size);
106        avgTime = ((avgTime * i) + (System.currentTimeMillis() - startTime))
107            / (i + 1);
108        binarySearchArray.verify(size, true);
109    }
110
111    System.out.println("Test successful, result is as follows:");
112    System.out.println("Average time for generation is: " + avgTime + " ms");
113 }
114
115 }
116
117
118 /* Session-Log:
119
120 ARRAYLIST based TEST
121 Please be patient, the following operations may take some time...
122 Test successful, result is as follows:
123 Average time for generation is: 5.16ms
124
125 */
126

```

```
1  /*
2   * OST - Uebungen 'Algorithmen & Datenstrukturen (AlgDat)'
3   * Version: Tue Sep 24 09:34:25 CEST 2024
4   */
5
6  package ex02.solution.task01;
7
8  import java.util.Collection;
9  import java.util.LinkedList;
10 import java.util.stream.Collectors;
11
12 public class BinarySearchTree<K extends Comparable<? super K>, V> {
13
14     protected Node root;
15
16     public static class Entry<K, V> {
17
18         private K key;
19         private V value;
20
21         public Entry(K key, V value) {
22             this.key = key;
23             this.value = value;
24         }
25
26         protected K setKey(K key) {
27             K oldKey = this.key;
28             this.key = key;
29             return oldKey;
30         }
31
32         public K getKey() {
33             return key;
34         }
35
36         public V setValue(V value) {
37             V oldValue = this.value;
38             this.value = value;
39             return oldValue;
40         }
41
42         public V getValue() {
43             return value;
44         }
45
46         @Override
47         public String toString() {
48             StringBuilder result = new StringBuilder();
49             result.append("[").append(key).append("/").append(value).append("]");
50             return result.toString();
51         }
52     } // End of class Entry
53
54     public class Node {
55
56         private Entry<K, V> entry;
57         private Node leftChild;
58         private Node rightChild;
59
60         public Node(Entry<K, V> entry) {
61             this.entry = entry;
62         }
63
64         public Node(Entry<K, V> entry, Node leftChild, Node rightChild) {
65             this.entry = entry;
66             this.leftChild = leftChild;
67             this.rightChild = rightChild;
68         }
69     }
```

```

70
71 public Entry<K, V> getEntry() {
72     return entry;
73 }
74
75 public Entry<K, V> setEntry(Entry<K, V> entry) {
76     Entry<K, V> oldEntry = entry;
77     this.entry = entry;
78     return oldEntry;
79 }
80
81 public Node getLeftChild() {
82     return leftChild;
83 }
84
85 public void setLeftChild(Node leftChild) {
86     this.leftChild = leftChild;
87 }
88
89 public Node getRightChild() {
90     return rightChild;
91 }
92
93 public void setRightChild(Node rightChild) {
94     this.rightChild = rightChild;
95 }
96
97 } // End of class Node
98
99
100 public Entry<K, V> insert(K key, V value) {
101     Entry<K, V> newEntry = new Entry<>(key, value);
102     root = insert(root, newEntry);
103     return newEntry;
104 }
105
106 protected Node insert(Node node, Entry<K, V> entry) {
107     if (node == null)
108         return newNode(entry);
109     else if (entry.getKey().compareTo(node.getEntry().getKey()) <= 0) {
110         node.leftChild = insert(node.leftChild, entry);
111     } else { /* if (entry.key > node.key) */
112         node.rightChild = insert(node.rightChild, entry);
113     }
114     return node;
115 }
116
117 /**
118  * Factory-Method: Creates a new node.
119  *
120  * @param entry
121  *     The entry to be inserted in the new node.
122  * @return The new created node.
123  */
124 protected Node newNode(Entry<K, V> entry) {
125     return new Node(entry);
126 }
127
128 public void clear() {
129     root = null;
130 }
131
132 public Entry<K, V> find(K key) {
133     Node result = find(root, key);
134     if (result == null) {
135         return null;
136     }
137     return result.getEntry();
138 }

```

```
139
140     protected Node find(Node node, K key) {
141         if (node == null) {
142             return null;
143         }
144         if (key.compareTo(node.getEntry().getKey()) < 0) {
145             return find(node.leftChild, key);
146         }
147         if (key.compareTo(node.getEntry().getKey()) > 0) {
148             return find(node.rightChild, key);
149         }
150         return node;
151     }
152
153     /**
154      * Returns a collection with all entries with key.
155      *
156      * @param key
157      *         The key to be searched.
158      * @return Collection of all entries found. An empty collection is returned if
159      *         no entry with key is found.
160      */
161     public Collection<Entry<K, V>> findAll(K key) {
162         Collection<Entry<K, V>> entries = new LinkedList<>();
163         findAll(root, key, entries);
164         return entries;
165     }
166
167     protected void findAll(Node node, K key, Collection<Entry<K, V>> entries) {
168         if (node == null) {
169             return;
170         }
171         if (key.compareTo(node.getEntry().getKey()) == 0) {
172             entries.add(node.getEntry());
173         }
174         if (key.compareTo(node.getEntry().getKey()) <= 0) {
175             findAll(node.leftChild, key, entries);
176         }
177         if (key.compareTo(node.getEntry().getKey()) >= 0) {
178             findAll(node.rightChild, key, entries);
179         }
180     }
181
182     /**
183      * Returns a collection with all entries in inorder.
184      *
185      * @return Inorder-Collection of all entries.
186      */
187     public Collection<Entry<K, V>> inorder() {
188         Collection<Node> inorderNodeList = new LinkedList<>();
189         inorder(root, inorderNodeList);
190         Collection<Entry<K, V>> inorderEntryList = inorderNodeList.stream()
191             .map(node -> node.getEntry()).collect(Collectors.toList());
192         return inorderEntryList;
193     }
194
195     protected void inorder(Node node, Collection<Node> inorderList) {
196         if (node == null)
197             return;
198         inorder(node.leftChild, inorderList);
199         inorderList.add(node);
200         inorder(node.rightChild, inorderList);
201     }
```

24.9.2024 9:34:25

BinarySearchTree.java

Page 4/7

```
202
203 /**
204  * Prints the entries of the tree as a list in inorder to the console.
205  */
206 public void printInorder() {
207     inorder().stream().forEach(e -> {
208         System.out.print(e + " ");
209     });
210     System.out.println();
211 }
212
213 public Entry<K, V> remove(Entry<K, V> entry) {
214     if (entry == null) {
215         return null;
216     }
217     RemoveResult result = remove(root, entry);
218     root = result.node;
219     return result.entry;
220 }
221
222 protected class RemoveResult {
223
224     private Node node;
225     private Entry<K, V> entry;
226
227     public RemoveResult(Node node, Entry<K, V> entry) {
228         this.node = node;
229         this.entry = entry;
230     }
231
232     RemoveResult set(Node node) {
233         this.node = node;
234         return this;
235     }
236
237     public Node getNode() {
238         return node;
239     }
240
241     public Entry<K, V> getEntry() {
242         return entry;
243     }
244
245 }
```

```

246
247     protected RemoveResult remove(final Node node, final Entry<K, V> entry) {
248         RemoveResult result = null;
249         if (node == null) {
250             return new RemoveResult(null, null);
251         }
252         if (entry.getKey().compareTo(node.getEntry().getKey()) < 0) {
253             result = remove(node.leftChild, entry);
254             node.leftChild = result.node;
255             return result.set(node);
256         } else if (entry.getKey().compareTo(node.getEntry().getKey()) > 0) {
257             result = remove(node.rightChild, entry);
258             node.rightChild = result.node;
259             return result.set(node);
260         } else {
261             // Key found: is this the correct entry?
262             if (node.getEntry() != entry) {
263                 // Searching for next entry with this key
264                 result = remove(node.leftChild, entry);
265                 node.leftChild = result.node;
266                 if (result.entry == null) {
267                     result = remove(node.rightChild, entry);
268                     node.rightChild = result.node;
269                 }
270                 return result.set(node);
271             }
272             // We have reached the correct node.
273             if (node.leftChild == null) {
274                 return new RemoveResult(node.rightChild, node.getEntry());
275             }
276             if (node.rightChild == null) {
277                 return new RemoveResult(node.leftChild, node.getEntry());
278             }
279             Entry<K, V> entryRemoved = node.getEntry();
280             Node q = getParentNext(node);
281             if (q == node) {
282                 node.setEntry(node.rightChild.getEntry());
283                 q.rightChild = q.rightChild.rightChild;
284             } else {
285                 node.setEntry(q.leftChild.getEntry());
286                 q.leftChild = q.leftChild.rightChild;
287             }
288             return new RemoveResult(node, entryRemoved);
289         }
290     }
291
292     /**
293     * Search for the inorder successor.
294     *
295     * @param p
296     *      The node for which the inorder successor shall be searched.
297     * @return The parent-node(!) of the inorder successor.
298     */
299     @SuppressWarnings("static-method")
300     protected Node getParentNext(Node p) {
301         if (p.rightChild.leftChild != null) {
302             p = p.rightChild;
303             while (p.leftChild.leftChild != null)
304                 p = p.leftChild;
305         }
306         return p;
307     }

```

```
308
309  /**
310   * The height of the tree.
311   *
312   * @return The current height. -1 for an empty tree.
313   */
314  public int getHeight() {
315      return getHeight(root);
316  }
317
318  protected int getHeight(Node p) {
319      if (p == null)
320          return -1;
321      int rHeight = getHeight(p.rightChild);
322      int lHeight = getHeight(p.leftChild);
323      return (lHeight < rHeight ? rHeight : lHeight) + 1;
324  }
325
326  public int size() {
327      return size(root);
328  }
329
330  protected int size(Node n) {
331      if (n == null) {
332          return 0;
333      }
334      return size(n.leftChild) + size(n.rightChild) + 1;
335  }
336
337  public boolean isEmpty() {
338      return size() == 0;
339  }
```



```
1  /*
2  * OST - Uebungen 'Algorithmen & Datenstrukturen (AlgDat)'
3  * Version: Mon Sep 30 16:38:18 CEST 2024
4  */
5
6  package ex03.solution.task03;
7
8  import java.util.Collection;
9
10 import ex02.solution.task01.BinarySearchTree.Entry;
11
12
13 public class AVLTree <K extends Comparable<? super K>, V> {
14
15     private AVLTreeImpl<K, V> avlTreeImpl =
16         new AVLTreeImpl<>();
17         //new AVLTreeImplADV<>("AVL-Tree"); // Show in ADV
18         //new AVLTreeImplADV<>("AVL-Tree", 1, 3); // Show in ADV: Be aware of NodeFixation
19         Exception!
20
21     public V put(K key, V value) {
22         return avlTreeImpl.put(key, value);
23     }
24
25     public V get(K key) {
26         return avlTreeImpl.get(key);
27     }
28
29     public int getHeight() {
30         return avlTreeImpl.getHeight();
31     }
32
33     public int size() {
34         return avlTreeImpl.size();
35     }
36
37     public boolean isEmpty() {
38         return avlTreeImpl.isEmpty();
39     }
40
41     public void clear() {
42         avlTreeImpl.clear();
43     }
44
45     public Collection<Entry<K, V>> inorder() {
46         return avlTreeImpl.inorder();
47     }
48
49     public void printInorder() {
50         avlTreeImpl.printInorder();
51     }
52
53     public void print() {
54         avlTreeImpl.print();
55     }
56
57     protected AVLTreeImpl<K, V> getImpl() {
58         return avlTreeImpl;
59     }
60 }
```

30.9.2024 16:38:18

AVLTreeImpl.java

Page 1/4

```

1  /*
2  * OST - Uebungen 'Algorithmen & Datenstrukturen (AlgDat)'
3  * Version: Mon Sep 30 16:38:18 CEST 2024
4  */
5
6  package ex03.solution.task03;
7
8  import java.util.Collection;
9  import java.util.LinkedList;
10 import java.util.List;
11
12 import ex02.solution.task01.BinarySearchTree;
13
14 class AVLTreeImpl<K extends Comparable<? super K>, V> extends
15     BinarySearchTree<K, V> {
16
17     /**
18      * After the BST-operation 'insert()':
19      * actionNode shall point to the parent of the new inserted node.
20      */
21     protected AVLNode actionNode;
22
23
24     protected class AVLNode extends BinarySearchTree<K, V>.Node {
25
26         private int height;
27         private Node parent;
28
29         AVLNode(Entry<K, V> entry) {
30             super(entry);
31         }
32
33         protected AVLNode setParent(AVLNode parent) {
34             AVLNode old = avlNode(this.parent);
35             this.parent = parent;
36             return old;
37         }
38
39         protected AVLNode getParent() {
40             return avlNode(parent);
41         }
42
43         protected int setHeight(int height) {
44             int old = this.height;
45             this.height = height;
46             return old;
47         }
48
49         protected int getHeight() {
50             return height;
51         }
52
53         @Override
54         public AVLNode getLeftChild() {
55             return avlNode(super.getLeftChild());
56         }
57
58         @Override
59         public AVLNode getRightChild() {
60             return avlNode(super.getRightChild());
61         }

```

```
62
63     @Override
64     public String toString() {
65         String result = String.format("%2d - %-6s : h=%d",
66                                     getEntry().getKey(), getEntry().getValue(), height);
67         if (parent == null) {
68             result += " ROOT";
69         } else {
70             boolean left = (parent.getLeftChild() == this) ? true : false;
71             result += (left ? " / " : " \\ ") + "parent(key)="
72                     + parent.getEntry().getKey();
73         }
74         return result;
75     }
76
77 } // End of class AVLNode
78
79
80 protected AVLNode getRoot() {
81     return avlNode(root);
82 }
83
84 public V put(K key, V value) {
85     Entry<K, V> entry = find(key);
86     if (entry != null) {
87         // key already exists in the Tree
88         return entry.setValue(value);
89     }
90     // key does not exist in the Tree yet
91     super.insert(key, value);
92     assureHeights(actionNode);
93     actionNode = null;
94     return null;
95 }
96
97 public V get(K key) {
98     Entry<K, V> entry = super.find(key);
99     if (entry == null) {
100         return null;
101     }
102     return entry.getValue();
103 }
104
105 @Override
106 protected Node insert(Node node, Entry<K, V> entry) {
107     if (node != null) {
108         actionNode = avlNode(node);
109     }
110     // calling now the BST-insert() which will do the work:
111     AVLNode result = avlNode(super.insert(node, entry));
112     if (node == null) {
113         // In this case: result of super.insert() is the new node!
114         result.setParent(actionNode);
115     }
116     return result;
117 }
```

```

118
119 /**
120  * The height of the tree.
121  *
122  * @return The current height. -1 for an empty tree.
123  */
124 @Override
125 public int getHeight() {
126     return height(avlNode(root));
127 }
128
129 /**
130  * Returns the height of this node.
131  *
132  * @param node
133  * @return The height or -1 if null.
134  */
135 @SuppressWarnings("static-method")
136 protected int height(AVLNode node) {
137     return (node != null) ? node.getHeight() : -1;
138 }
139
140 /**
141  * Assures the heights of the tree from 'node' up to the root.
142  *
143  * @param node
144  *         The node from where to start.
145  */
146 protected void assureHeights(AVLNode node) {
147     while (node != null) {
148         setHeight(node);
149         node = node.getParent();
150     }
151 }
152
153 /**
154  * Assures the correct height for node.
155  *
156  * @param node
157  *         The node to assure its height.
158  */
159 protected void setHeight(AVLNode node) {
160     if (node == null) {
161         return;
162     }
163     int heightLeftChild = height(node.getLeftChild());
164     int heightRightChild = height(node.getRightChild());
165     node.setHeight(1 + Math.max(heightLeftChild, heightRightChild));
166 }
167
168 /**
169  * Factory-Method. Creates a new node.
170  *
171  * @param entry
172  *         The entry to be inserted in the new node.
173  * @return The new created node.
174  */
175 @Override
176 protected Node newNode(Entry<K, V> entry) {
177     return new AVLNode(entry);
178 }

```

```
179
180  @Override
181  protected void inorder(Node node, Collection<Node> inorderList) {
182      super.inorder(node, inorderList);
183  }
184
185  // Type-Casting: Node -> AVLNode (Cast-Encapsulation)
186  @SuppressWarnings({ "unchecked", "static-method" })
187  protected AVLNode avlNode(Node node) {
188      return (AVLNode)node;
189  }
190
191  public void print() {
192      List<Node> nodeList = new LinkedList<>();
193      inorder(root, nodeList);
194      for (Node node: nodeList) {
195          System.out.println(node + " ");
196      }
197  }
198
199  }
200
201
```

```
1  /*
2   * OST - Uebungen 'Algorithmen & Datenstrukturen (AlgDat)'
3   * Version: Sun Oct 6 13:58:27 CEST 2024
4   */
5
6  package ex04.solution.task01;
7
8  import java.util.Collection;
9
10 import ex02.solution.task01.BinarySearchTree.Entry;
11
12
13 public class AVLTree <K extends Comparable<? super K>, V> {
14
15     private AVLTreeImpl<K, V> avlTreeImpl =
16         new AVLTreeImpl<>();
17         //new AVLTreeImplADV<K, V>("AVL-Tree Ü4"); // Show in ADV
18         //new AVLTreeImplADV<K, V>("AVL-Tree Ü4", 3, 2); // Show in ADV: Fix heights
19
20     public V put(K key, V value) {
21         return avlTreeImpl.put(key, value);
22     }
23
24     public V get(K key) {
25         return avlTreeImpl.get(key);
26     }
27
28     public V remove(K key) {
29         return avlTreeImpl.remove(key);
30     }
31
32     public int getHeight() {
33         return avlTreeImpl.getHeight();
34     }
35
36     public int size() {
37         return avlTreeImpl.size();
38     }
39
40     public boolean isEmpty() {
41         return avlTreeImpl.isEmpty();
42     }
43
44     public void clear() {
45         avlTreeImpl.clear();
46     }
47
48     public Collection<Entry<K, V>> inorder() {
49         return avlTreeImpl.inorder();
50     }
51
52     public void printInorder() {
53         avlTreeImpl.printInorder();
54     }
55
56     public void print() {
57         avlTreeImpl.print();
58     }
59
60     protected AVLTreeImpl<K, V> getImpl() {
61         return avlTreeImpl;
62     }
63 }
```

6.10.2024 13:58:27

AVLTreeImpl.java

Page 1/6

```

1  /*
2   * OST - Uebungen 'Algorithmen & Datenstrukturen (AlgDat)'
3   * Version: Sun Oct 6 13:58:27 CEST 2024
4   */
5
6  package ex04.solution.task01;
7
8  import java.lang.reflect.Array;
9  import java.util.Collection;
10 import java.util.LinkedList;
11 import java.util.List;
12
13 import ex02.solution.task01.BinarySearchTree;
14
15
16 class AVLTreeImpl<K extends Comparable<? super K>, V> extends
17     BinarySearchTree<K, V> {
18
19     /**
20      * After a BST-operation, actionNode shall point to where the balance has to
21      * be checked. -> rebalance() will then be called with actionNode.
22      */
23     protected AVLNode actionNode;
24
25
26     protected class AVLNode extends BinarySearchTree<K, V>.Node {
27
28         private int height;
29         private Node parent;
30
31         AVLNode(Entry<K, V> entry) {
32             super(entry);
33         }
34
35         protected AVLNode setParent(AVLNode parent) {
36             AVLNode old = avlNode(this.parent);
37             this.parent = parent;
38             return old;
39         }
40
41         protected AVLNode getParent() {
42             return avlNode(parent);
43         }
44
45         protected int setHeight(int height) {
46             int old = this.height;
47             this.height = height;
48             return old;
49         }
50
51         protected int getHeight() {
52             return height;
53         }
54
55         @SuppressWarnings("unchecked")
56         @Override
57         public void setLeftChild(BinarySearchTree<K, V>.Node leftChild) {
58             super.setLeftChild(leftChild);
59             if (leftChild != null) {
60                 ((AVLNode) leftChild).setParent(this);
61             }
62         }
63
64         @Override
65         public AVLNode getLeftChild() {
66             return avlNode(super.getLeftChild());
67         }
68     }

```

```

68
69     @SuppressWarnings("unchecked")
70     @Override
71     public void setRightChild(BinarySearchTree<K, V>.Node rightChild) {
72         super.setRightChild(rightChild);
73         if (rightChild != null) {
74             ((AVLNode) rightChild).setParent(this);
75         }
76     }
77
78     @Override
79     public AVLNode getRightChild() {
80         return avlNode(super.getRightChild());
81     }
82
83     @Override
84     public String toString() {
85         String result = String.format("%2d - %-6s : h=%d",
86                                     getEntry().getKey(), getEntry().getValue(), height);
87         if (parent == null) {
88             result += " ROOT";
89         } else {
90             boolean left = (parent.getLeftChild() == this) ? true : false;
91             result += (left ? " / " : " \\ ") + "parent(key)="
92                 + parent.getEntry().getKey();
93         }
94         return result;
95     }
96
97 } // End of class AVLNode
98
99
100 protected AVLNode getRoot() {
101     return avlNode(root);
102 }
103
104 public V put(K key, V value) {
105     Entry<K, V> entry = find(key);
106     if (entry != null) {
107         // key already exists in the Tree
108         return entry.setValue(value);
109     }
110     // key does not exist in the Tree yet
111     super.insert(key, value);
112     rebalance(actionNode);
113     actionNode = null;
114     return null;
115 }
116
117 public V get(K key) {
118     Entry<K, V> entry = super.find(key);
119     if (entry == null) {
120         return null;
121     }
122     return entry.getValue();
123 }

```



```

124
125 @Override
126 protected Node insert(Node node, Entry<K, V> entry) {
127     if (node != null) {
128         actionNode = avlNode(node);
129     }
130     // calling now the BST-insert() which will do the work:
131     AVLNode result = avlNode(super.insert(node, entry));
132     if (node == null) {
133         // In this case: result of super.insert() is the new node!
134         result.setParent(actionNode);
135     }
136     return result;
137 }
138
139 /**
140  * The height of the tree.
141  *
142  * @return The current height. -1 for an empty tree.
143  */
144 @Override
145 public int getHeight() {
146     return height(avlNode(root));
147 }
148
149 /**
150  * Returns the height of this node.
151  *
152  * @param node
153  * @return The height or -1 if null.
154  */
155 @SuppressWarnings("static-method")
156 protected int height(AVLNode node) {
157     return (node != null) ? node.getHeight() : -1;
158 }
159
160 /**
161  * Restructures the tree with rotations.
162  *
163  * @param xPos
164  *         The X-node.
165  * @return The new root-node of this subtree.
166  */
167 protected AVLNode restructure(AVLNode xPos) {
168     AVLNode yPos = xPos.getParent();
169     AVLNode zPos = yPos.getParent();
170     AVLNode newSubTreeRoot = null;
171     if (yPos == zPos.getLeftChild()) {
172         if (xPos == yPos.getLeftChild()) {
173             newSubTreeRoot = rotateWithLeftChild(zPos);
174         } else {
175             newSubTreeRoot = doubleRotateWithLeftChild(zPos);
176         }
177     } else {
178         if (xPos == yPos.getRightChild()) {
179             newSubTreeRoot = rotateWithRightChild(zPos);
180         } else {
181             newSubTreeRoot = doubleRotateWithRightChild(zPos);
182         }
183     }
184     return newSubTreeRoot;
185 }

```

```

186
187     protected AVLNode tallerChild(AVLNode node) {
188         AVLNode result;
189         if (height(node.getLeftChild()) >= height(node.getRightChild())) {
190             result = node.getLeftChild();
191         } else {
192             result = node.getRightChild();
193         }
194         return result;
195     }
196
197     protected AVLNode rotateWithLeftChild(AVLNode k2) {
198         AVLNode parentSubtree = k2.getParent();
199         AVLNode k1 = k2.getLeftChild();
200         k2.setLeftChild(k1.getRightChild());
201         k1.setRightChild(k2);
202         adjustSubtreeParent(k2, k1, parentSubtree);
203         return k1;
204     }
205
206     protected AVLNode doubleRotateWithLeftChild(AVLNode k3) {
207         //k3.setLeftChild(rotateWithRightChild(k3.getLeftChild()));
208         // -> k3.setLeftChild() is done/ensured in adjustSubtreeParent():
209         rotateWithRightChild(k3.getLeftChild());
210         return rotateWithLeftChild(k3);
211     }
212
213     protected AVLNode rotateWithRightChild(AVLNode k1) {
214         AVLNode parentSubtree = k1.getParent();
215         AVLNode k2 = k1.getRightChild();
216         k1.setRightChild(k2.getLeftChild());
217         k2.setLeftChild(k1);
218         adjustSubtreeParent(k1, k2, parentSubtree);
219         return k2;
220     }
221
222     protected AVLNode doubleRotateWithRightChild(AVLNode k3) {
223         //k3.setRightChild(rotateWithLeftChild(k3.getRightChild()));
224         // -> k3.setRightChild() is done/ensured in adjustSubtreeParent():
225         rotateWithLeftChild(k3.getRightChild());
226         return rotateWithRightChild(k3);
227     }
228
229     /**
230      * Assures the connection between a restructured subtree and the parent of
231      * this subtree.
232      * Used after rotations.
233      *
234      * @param oldSubtreeRoot
235      *        The old root-node of this subtree.
236      * @param newSubtreeRoot
237      *        The new root-node of this subtree.
238      * @param parentSubtree
239      *        The parent-node of this subtree.
240      */
241     protected void adjustSubtreeParent(AVLNode oldSubtreeRoot,
242         AVLNode newSubtreeRoot, AVLNode parentSubtree) {
243         if (parentSubtree != null) {
244             if (parentSubtree.getLeftChild() == oldSubtreeRoot) {
245                 parentSubtree.setLeftChild(newSubtreeRoot);
246             } else {
247                 parentSubtree.setRightChild(newSubtreeRoot);
248             }
249         } else { // newSubtreeRoot is now also the root of the whole tree
250             root = newSubtreeRoot;
251             newSubtreeRoot.setParent(null);
252         }
253     }

```

```

254
255     protected boolean isBalanced(AVLNode node) {
256         int bf = height(node.getLeftChild()) - height(node.getRightChild());
257         return (-1 <= bf) && (bf <= 1);
258     }
259
260     /**
261      * Assures the balance of the tree from 'node' up to the root.
262      *
263      * @param node
264      *     The node from where to start.
265      */
266     protected void rebalance(AVLNode node) {
267         while (node != null) {
268             setHeight(node);
269             if (!isBalanced(node)) {
270                 AVLNode xPos = tallerChild(tallerChild(node));
271                 node = restructure(xPos);
272                 setHeight(node.getLeftChild());
273                 setHeight(node.getRightChild());
274                 setHeight(node);
275             }
276             node = node.getParent();
277         }
278     }
279
280     /**
281      * Assures the correct height for node.
282      *
283      * @param node
284      *     The node to assure its height.
285      */
286     protected void setHeight(AVLNode node) {
287         if (node == null) {
288             return;
289         }
290         int heightLeftChild = height(node.getLeftChild());
291         int heightRightChild = height(node.getRightChild());
292         node.setHeight(1 + Math.max(heightLeftChild, heightRightChild));
293     }
294
295     /**
296      * Factory-Method. Creates a new node.
297      *
298      * @param entry
299      *     The entry to be inserted in the new node.
300      * @return The new created node.
301      */
302     @Override
303     protected Node newNode(Entry<K, V> entry) {
304         return new AVLNode(entry);
305     }
306
307     public V remove(K key) {
308         Entry<K, V> entry = find(key);
309         if (entry == null) {
310             return null;
311         }
312         // calling now the BST-remove(Entry) which will do the work:
313         super.remove(entry);
314         if (actionNode != null) {
315             assureParentForChilds(actionNode);
316             rebalance(actionNode);
317             actionNode = null;
318         }
319         return entry.getValue();
320     }

```

```
321
322 @Override
323 protected RemoveResult remove(Node node, Entry<K, V> entry) {
324     if (node.getEntry() == entry) {
325         actionNode = avlNode(node).getParent();
326     }
327     // calling now the BST-remove(Node, Entry) which will do the work:
328     return super.remove(node, entry);
329 }
330
331 @Override
332 protected Node getParentNext(Node p) {
333     actionNode = avlNode(super.getParentNext(p));
334     return actionNode;
335 }
336
337 @SuppressWarnings("static-method")
338 protected void assureParentForChilds(AVLNode parent) {
339     @SuppressWarnings("unchecked")
340     AVLNode[] childs = (AVLNode[])Array.newInstance(parent.getClass(), 2);
341     childs[0] = parent.getLeftChild();
342     childs[1] = parent.getRightChild();
343     for (AVLNode child : childs) {
344         if (child != null) {
345             child.setParent(parent);
346         }
347     }
348 }
349
350 @Override
351 protected void inorder(Node node, Collection<Node> inorderList) {
352     super.inorder(node, inorderList);
353 }
354
355 // Type-Casting: Node -> AVLNode (Cast-Encapsulation)
356 @SuppressWarnings({ "unchecked", "static-method" })
357 protected AVLNode avlNode(Node node) {
358     return (AVLNode)node;
359 }
360
361 public void print() {
362     List<Node> nodeList = new LinkedList<>();
363     inorder(root, nodeList);
364     for (Node node: nodeList) {
365         System.out.println(node + " ");
366     }
367 }
368
369 }
370
371
```

```
1  /**
2   * OST - Uebungen 'Algorithmen & Datenstrukturen (AlgDat)'
3   * Version: Tue Oct 15 12:38:16 CEST 2024
4   */
5
6  package ex05.solution.task01;
7
8  import java.util.Arrays;
9  import java.util.Random;
10
11  /**
12   * @author tbeeler
13   *
14   * BubbleSort. Two versions of the bubblesort for sorting integers.
15   *
16   */
17  public class BubbleSort {
18
19      /**
20       * First version: no optimization.
21       *
22       * @param <T>
23       *         Type of elements to be sorted. Must be comparable.
24       * @param sequence
25       *         The sequence to be sorted.
26       */
27      public static <T extends Comparable<? super T>> void bubbleSort1(T[] sequence) {
28          boolean bubbled = false;
29          T temp;
30          do {
31              bubbled = false;
32              for (int i = 1; i < sequence.length; i++) {
33                  if (sequence[i].compareTo(sequence[i - 1]) < 0) {
34                      // swap
35                      temp = sequence[i];
36                      sequence[i] = sequence[i - 1];
37                      sequence[i - 1] = temp;
38                      bubbled = true;
39                  }
40              } while (bubbled);
41          }
42      }
43
44      /**
45       * Second version with slight optimization: The upper boundary is reduced by
46       * one in every iteration (the biggest bubble is on top now).
47       *
48       * @param <T>
49       *         Type of elements to be sorted. Must be comparable.
50       * @param sequence
51       *         The sequence to be sorted.
52       */
53      public static <T extends Comparable<? super T>> void bubbleSort2(T[] sequence) {
54          boolean bubbled = false;
55          T temp;
56          int unsortedSize = sequence.length;
57          do {
58              bubbled = false;
59              for (int i = 1; i < unsortedSize; i++) {
60                  if (sequence[i].compareTo(sequence[i - 1]) < 0) {
61                      // swap
62                      temp = sequence[i];
63                      sequence[i] = sequence[i - 1];
64                      sequence[i - 1] = temp;
65                      bubbled = true;
66                  }
67              }
68              unsortedSize--;
69          } while (bubbled);
70      }
71  }
```

```

1  /*
2   * OST - Uebungen 'Algorithmen & Datenstrukturen (AlgDat)'
3   * Version: Tue Oct 15 12:39:35 CEST 2024
4   */
5
6  package ex05.solution.task02;
7
8  import java.lang.reflect.Array;
9  import java.util.Random;
10
11 public class MergeSort {
12
13     /**
14      * Sorts an Array with the Merge-Sort Algorithm.
15      * Precondition: Length must be 2^x.
16      * @param s Sequence (Array) to be sorted.
17      * @return The sorted Sequence (Array).
18      */
19     public static <T extends Comparable<? super T>> T[] mergeSort(T[] s) {
20         int n = s.length;
21         if (n > 1) {
22             Partitions<T> s12 = partition(s, n/2);
23             T[] s1 = mergeSort(s12.s1);
24             T[] s2 = mergeSort(s12.s2);
25             s = merge(s1, s2);
26         }
27         return s;
28     }
29
30     record Partitions<T>(T[] s1, T[] s2) {}
31
32     static <T> Partitions<T> partition(T[] s, int length) {
33         T[] s1 = newInstance(s, length);
34         T[] s2 = newInstance(s, length);
35         System.arraycopy(s, 0, s1, 0, length);
36         System.arraycopy(s, length, s2, 0, length);
37         return new Partitions<T>(s1, s2);
38     }
39
40     /**
41      * Merges the two Sequences (Arrays) 'a' and 'b' in ascending Order.
42      * @param a Sequence A.
43      * @param b Sequence B.
44      * @return The merged Sequence.
45      */
46     static <T extends Comparable<? super T>> T[] merge(T[] a, T[] b) {
47         T[] s = newInstance(a, a.length * 2);
48         int ai = 0; // First Element in 'Sequence' A
49         int bi = 0; // First Element in 'Sequence' B
50         int si = 0; // Last Element in 'Sequence' S
51         while (!(ai == a.length) && !(bi == b.length)) {
52             if (a[ai].compareTo(b[bi]) < 0) {
53                 s[si++] = a[ai++];
54             }
55             else {
56                 s[si++] = b[bi++];
57             }
58         }
59         while (!(ai == a.length)) {
60             s[si++] = a[ai++];
61         }
62         while (!(bi == b.length)) {
63             s[si++] = b[bi++];
64         }
65         return s;
66     }

```

15.10.2024 12:39:35

MergeSort.java

Page 2/3

```

67  /**
68  * Utility-Method to create a <T>-Array.
69  *
70  *
71  * @param array
72  *      An Array with the same Type as the new one (only used to get the
73  *      correct Type for the new Array).
74  * @param length
75  *      The Length of the new Array.
76  * @return The new created Array.
77  */
78  @SuppressWarnings("unchecked")
79  static <T> T[] newInstance(T[] array, int length) {
80      return (T[]) Array.newInstance(array[0].getClass(), length);
81  }
82
83
84  public static void main(String[] args) {
85
86      Integer[] array = {7, 2, 9, 4, 3, 8, 6, 1};
87      Integer[] originalArray = array.clone();
88      printArray(array);
89
90      array = mergeSort(array);
91
92      printArray(array);
93      verify(originalArray, array);
94
95      /* Making some Test to measure the Time needed of mergeSort().
96       * Creating int-Arrays, beginning with Length of 2^minExponent
97       * until the last Array with Length of 2^maxExponent.
98       */
99      final int minExponent = 10;
100     final int maxExponent = 15;
101     int n = (int) Math.round(Math.pow(2, maxExponent));
102     array = new Integer[n];
103     Random rand = new Random(0);    // a Random-Generator
104     for (int i = 0; i < n; i++) {
105         array[i] = rand.nextInt(101); // generating Numbers: 0..100
106     }
107     long lastTime = Long.MAX_VALUE;
108     for (int exp = minExponent; exp <= maxExponent; exp++) {
109         int len = (int) Math.round(Math.pow(2, exp));
110         Integer[] arr = new Integer[len];
111         final int MEASUREMENTS = 10;
112         long minTime = Long.MAX_VALUE;
113         for (int m = 0; m < MEASUREMENTS; m++) {
114             System.arraycopy(array, 0, arr, 0, len);
115             long start = System.nanoTime();
116             arr = mergeSort(arr);
117             long end = System.nanoTime();
118             long time = end - start;
119             if (time < minTime) {
120                 minTime = time;
121             }
122             verify(array, arr);
123         }
124         System.out.format("Array-Size: %,7d          Time: %,6.1f ms      "
125             + "Ratio to last: %2.1f\n",
126             len, (double) minTime / (long) 1e6,
127             (double) minTime / lastTime);
128         lastTime = minTime;
129     }
130 }

```



```

71  enum SequenceType {RANDOM, EQUAL, SORTED}
72
73
74  public static void main(String[] args) throws Exception {
75      Comparator<Point> comp = new PointComparator();
76      QuickSort qs = new QuickSort();
77      Random random = new Random(0);
78      int nSequence = 50;
79      if (args.length > 0)
80          nSequence = Integer.parseInt(args[0]);
81      final Point s1[] = new Point[nSequence];
82      for (int i = 0; i < s1.length; i++) {
83          s1[i] = new Point((int)(random.nextDouble() * 100),
84                          (int)(random.nextDouble() * 100));
85      }
86      printArray(s1);
87      System.out.print("Quick sort...");
88      long then = System.currentTimeMillis();
89      qs.inPlaceQuickSort(s1, comp, 0, nSequence - 1);
90      long now = System.currentTimeMillis();
91      long d1 = now - then;
92      System.out.println("done.");
93      printArray(s1);
94      System.out.println("Time quick sort [ms]: " + d1);
95
96      System.out.println("\nRuntime:");
97      final int INITIAL_SIZE = 64;
98      final int NR_OF_DOUBLING = 4;
99      for (SequenceType seqType: SequenceType.values()) {
100         System.out.println("Sequence-Type: "+seqType);
101         int arraySize = INITIAL_SIZE;
102         long lastTime = Long.MAX_VALUE;
103         for (int j = 0; j <= NR_OF_DOUBLING; j++) {
104             final int MEASUREMENTS = 200;
105             long minimalTime = Long.MAX_VALUE;
106             Point[] s2 = new Point[arraySize];
107             for (int m = 0; m < MEASUREMENTS; m++) {
108                 for (int i = 0; i < s2.length; i++) {
109                     switch (seqType) {
110                         case RANDOM:
111                             s2[i] = new Point((int)(Math.random() * 100),
112                                                 (int)(Math.random() * 100));
113                             break;
114                         case EQUAL:
115                             s2[i] = new Point(1, 1);
116                             break;
117                         case SORTED:
118                             s2[i] = new Point(i, i);
119                             break;
120                         default:
121                             throw new Exception("Unexpected SequenceType: " + seqType.name());
122                     }
123                 }
124                 long startTime = System.nanoTime();
125                 qs.inPlaceQuickSort(s2, comp, 0, arraySize - 1);
126                 long endTime = System.nanoTime();
127                 long time = endTime - startTime;
128                 if (time < minimalTime) {
129                     minimalTime = time;
130                 }
131                 Point[] test = s2.clone();
132                 Arrays.sort(test, comp);
133                 if (!Arrays.equals(s2, test)) {
134                     System.err.println("ERROR:");
135                     System.err.println(Arrays.toString(s2));
136                     System.err.println(Arrays.toString(test));
137                     System.exit(1);
138                 }
139             }

```

```
140
141     System.out.format(
142         "Array-Size: %6d   Time: %,7.0f us   Ratio to last: %2.1f\n",
143         arraySize, (double) minimalTime / (long) 1e3,
144         (double) minimalTime / lastTime);
145     lastTime = minimalTime;
146     arraySize = arraySize * 2;
147 }
148 }
149 }
150
151 private static void printArray(final Point[] array) {
152     if (array.length > 300) {
153         System.out.println("Too many elements, not printing to stdout.");
154     } else {
155         for (Point point: array) {
156             System.out.print("(" + point.x + "/" + point.y + "), ");
157         }
158         System.out.println();
159     }
160 }
161
162 }
163
164
165 class PointComparator implements Comparator<Point> {
166
167     /**
168      * Total order relation for points:
169      * p1 > p2   | p1.x > p2.x
170      * p1 > p2   | p1.x = p2.x && p1.y > p2.y
171      * p1 = p2   | p1.x = p2.x && p1.y = p2.y
172      * else p1 < p2
173      *
174      * @return p1 > p2   : +1,
175      *         p1 == p2 : 0,
176      *         p1 < p2  : -1
177      *
178      * @author tbeeler
179      */
180     @Override
181     public int compare(Point p1, Point p2) {
182         if (p1.x < p2.x)
183             return -1;
184         if (p1.x == p2.x) {
185             if (p1.y == p2.y)
186                 return 0;
187             if (p1.y < p2.y)
188                 return -1;
189         }
190         return 1;
191     }
192 }
193 }
```

```
1  /*
2  * OST - Uebungen 'Algorithmen & Datenstrukturen (AlgDat)'
3  * Version: Tue Oct 22 15:38:42 CEST 2024
4  */
5
6  package ex06.solution.task02;
7
8  import java.util.Arrays;
9  import java.util.LinkedList;
10 import java.util.List;
11 import java.util.OptionalInt;
12 import java.util.concurrent.atomic.AtomicInteger;
13 import java.util.stream.IntStream;
14
15 /**
16  * A Radix-Sort which uses internally a Bucket-Sort to sort a list of arrays of
17  * strings.
18  *
19  * @author mbuehlma
20  *
21  */
22 public class RadixSort {
23
24     // buckets used for bucket sort
25     private final LinkedList<String>[] buckets;
26
27     @SuppressWarnings("unchecked")
28     RadixSort() {
29         // create LinkedList for buckets
30         buckets = (LinkedList<String>[]) new LinkedList<?>[1 + ('z' - 'a' + 1)];
31         //for (int i = 0; i < buckets.length; i++) {
32         //    buckets[i] = new LinkedList<String>();
33         //}
34         IntStream.range(0, buckets.length).parallel()
35             .forEach(i -> buckets[i] = new LinkedList<>());
36         // alternative: Arrays.parallelSetAll(buckets, i -> new LinkedList<>());
37     }
38
39     public void radixSort(String[] data) {
40
41         // find max string-length
42         int maxLength = -1;
43         OptionalInt max = Arrays.stream(data).mapToInt(str -> str.length()).max();
44         if (max.isPresent()) {
45             maxLength = max.getAsInt();
46         } else {
47             return;
48         }
49
50         // bucketsort from max index to first index
51         for (int i = maxLength - 1; i >= 0; i--) {
52             bucketSort(data, i);
53         }
54     }
55 }
```

```
56
57 protected void bucketSort(String[] data, int index) {
58
59     // insert data elements to buckets
60     //for (String str: data) {
61     //    if (str.length() <= index) {
62     //        buckets[0].addLast(str);
63     //    } else {
64     //        buckets[str.charAt(index) - 'a' + 1].addLast(str);
65     //    }
66     //}
67     Arrays.stream(data).forEachOrdered(str -> {
68         if (str.length() <= index) {
69             buckets[0].addLast(str);
70         } else {
71             buckets[str.charAt(index) - 'a' + 1].addLast(str);
72         }
73     });
74
75     // shift bucket elements back into data array
76     //int n = 0; // current index in data-array.
77     //for (int i = 0; i < buckets.length; i++) {
78     //    while (!buckets[i].isEmpty()) {
79     //        data[n++] = buckets[i].removeFirst();
80     //    }
81     //}
82     String[] phase2Array = Arrays.asList(buckets).stream().flatMap(List::stream)
83         .toArray(String[]::new);
84     System.arraycopy(phase2Array, 0, data, 0, phase2Array.length);
85     Arrays.stream(buckets).parallel().forEach(list -> list.clear());
86
87 }
```

```
1  /*
2   * OST - Uebungen 'Algorithmen & Datenstrukturen (AlgDat)'
3   * Version: Tue Nov 5 10:53:45 CET 2024
4   */
5
6  package ex08.solution.task02;
7
8  import java.util.Iterator;
9
10 public interface Multimap<K, V> {
11
12     /**
13      * Inserts a key/value pair into the multimap.
14      */
15     void insert(K key, V value);
16
17     /**
18      * Returns the first value for a given key.
19      * If key is not found: null
20      */
21     V find(K key);
22
23     /**
24      * Returns all values for a given key.
25      * @return Iterator over all values.
26      *         If key is not found: Iterator without next.
27      */
28     Iterator<V> findAll(K key);
29
30     /**
31      * Removes all values for a given key.
32      */
33     void remove(K key);
34
35     /**
36      * Returns the number of values in the trie.
37      */
38     int size();
39
40 }
41
42
43
```

5.11.2024 10:53:45

TrieNode.java

Page 1/1

```
1  /*
2   * OST - Uebungen 'Algorithmen & Datenstrukturen (AlgDat)'
3   * Version: Tue Nov  5 10:53:45 CET 2024
4   */
5
6  package ex08.solution.task02;
7
8  import java.util.LinkedList;
9  import java.util.List;
10
11  class TrieNode<V> {
12
13      private String keySubstr;
14      private LinkedList<V> values;
15      private LinkedList<TrieNode<V>> childs;
16
17      public TrieNode() {
18          this.keySubstr = "";
19          this.values = new LinkedList<>();
20          this.childs = new LinkedList<>();
21      }
22
23      public String getKeySubstr() {
24          return keySubstr;
25      }
26
27      public void setKeySubstr(String keySubstr) {
28          this.keySubstr = keySubstr;
29      }
30
31      public List<TrieNode<V>> getChilds() {
32          return childs;
33      }
34
35      public List<V> getValues() {
36          return values;
37      }
38  }
39
40
41
42
```

```
1  /**
2   * OST - Uebungen 'Algorithmen & Datenstrukturen (AlgDat)'
3   * Version: Tue Nov 5 10:53:45 CET 2024
4   */
5
6  package ex08.solution.task02;
7
8  import java.util.Iterator;
9  import java.util.LinkedList;
10 import java.util.stream.Collectors;
11
12 public class TrieMultimap<V> implements Multimap<String, V> {
13
14     private TrieNode<V> root;
15     private int size;
16
17     private enum Modification {
18         INSERT, REMOVE
19     }
20
21     public TrieMultimap() {
22         this.root = new TrieNode<>();
23     }
24
25     /**
26     * Inserts a key/value pair into the multimap.
27     */
28     @Override
29     public void insert(String key, V value) {
30         TrieNode<V> result = find(root, key);
31         if (result != null) {
32             result.getValues().add(value);
33         } else {
34             modify(Modification.INSERT, root, key, 0, value);
35         }
36         size++;
37     }
38
39     /**
40     * Returns the first value for a given key.
41     * null if key is not found.
42     */
43     @Override
44     public V find(String key) {
45         TrieNode<V> result = find(root, key);
46         if (result != null) {
47             return result.getValues().get(0);
48         } else {
49             return null;
50         }
51     }
52
53     /**
54     * Returns all values for a given key.
55     *
56     * @return Iterator over all values.
57     *         If key is not found: Iterator without next.
58     */
59     @Override
60     public Iterator<V> findAll(String key) {
61         TrieNode<V> result = find(root, key);
62         if (result != null) {
63             return result.getValues().iterator();
64         } else {
65             return new LinkedList<V>().iterator();
66         }
67     }
68 }
```

5.11.2024 10:53:45

TrieMultimap.java

Page 2/6

```

68  /**
69  * Searches for a match for keySuffix in all childs (excluding current node).
70  * @param node The node to start from.
71  * @param keySuffix The remaining key-suffix to search for in the childs.
72  * @return The node with an exact match.
73  *         Otherwise null.
74  */
75  */
76  private TrieNode<V> find(TrieNode<V> node, String keySuffix) {
77      if (keySuffix.length() == 0) {
78          return node;
79      }
80      for (TrieNode<V> child : node.getChilds()) {
81          if (keySuffix.startsWith(child.getKeySubstr())) {
82              keySuffix = keySuffix.substring(child.getKeySubstr().length());
83              return find(child, keySuffix);
84          }
85      }
86      return null;
87  }
88
89  private boolean modify(Modification operation, TrieNode<V> node, String key,
90                        int keyIndex, V value) {
91      if (key.length() == keyIndex) {
92          // found the node!
93          if (operation == Modification.INSERT) {
94              node.getValues().add(value);
95          } else { // REMOVE
96              size -= node.getValues().size();
97              node.getValues().clear();
98          }
99          return true;
100      }
101      for (TrieNode<V> child : node.getChilds()) {
102          if (child.getKeySubstr().charAt(0) == key.charAt(keyIndex)) {
103              child = decompress(node, child);
104              boolean result = modify(operation, child, key, ++keyIndex, value);
105              compress(node, child);
106              return result;
107          }
108      }
109      // there is no corresponding child:
110      if (operation == Modification.INSERT) {
111          TrieNode<V> newNode = new TrieNode<>();
112          newNode.setKeySubstr(key.substring(keyIndex, keyIndex + 1));
113          node.getChilds().add(newNode);
114          modify(Modification.INSERT, newNode, key, ++keyIndex, value);
115          compress(node, newNode);
116      } else { // REMOVE
117          return false; // not found
118      }
119      return false;
120  }

```



```

121
122 /**
123  * Decompresses the child.
124  * @param node The current node.
125  * @param child The child which shall be decompressed.
126  * @return The new node which represents the child-position (with only one char as
127  * keySubstr) after the decompression.
128  */
129 private TrieNode<V> decompress(TrieNode<V> node, TrieNode<V> child) {
130     if (child.getKeySubstr().length() == 1) {
131         return child; // no decompression necessary
132     }
133     // insert an additional, single-char node (de-compressing):
134     TrieNode<V> newChild = new TrieNode<>();
135     newChild.setKeySubstr(child.getKeySubstr().substring(0, 1));
136     child.setKeySubstr(child.getKeySubstr().substring(1));
137     newChild.getChilds().add(child);
138     node.getChilds().add(newChild);
139     node.getChilds().remove(child);
140     return newChild;
141 }
142
143 /**
144  * Compresses the trie below child if possible or deletes child if it has to be
145  * deleted (because of remove()).
146  * @param node The current node.
147  * @param child The child which shall be compressed or deleted.
148  */
149 private void compress(TrieNode<V> node, TrieNode<V> child) {
150     if (child.getValues().isEmpty() && (child.getChilds().size() == 1)) {
151         // compress:
152         TrieNode<V> childOfChild = child.getChilds().get(0);
153         child.setKeySubstr(child.getKeySubstr().concat(childOfChild.getKeySubstr()));
154         child.getValues().addAll(childOfChild.getValues());
155         child.getChilds().addAll(childOfChild.getChilds());
156         child.getChilds().remove(childOfChild);
157         return;
158     }
159     if (child.getChilds().isEmpty() && (child.getValues().isEmpty())) {
160         // this is a removed node:
161         node.getChilds().remove(child);
162         return;
163     }
164 }
165
166 /**
167  * Removes all values for a given key.
168  */
169 @Override
170 public void remove(String key) {
171     TrieNode<V> result = find(root, key);
172     if (result != null) {
173         modify(Modification.REMOVE, root, key, 0, null);
174     } else {
175         return;
176     }
177 }
178
179 /**
180  * Returns the number of values in the trie.
181  */
182 @Override
183 public int size() {
184     assert size(root) == size : "size()=" + size(root) + " size="+size;
185     return size;
186 }

```

5.11.2024 10:53:45

TrieMultimap.java

Page 4/6

```

187
188 /**
189  * @return Number of values in this node and its child nodes.
190  */
191 private int size(TrieNode<V> element) {
192     int countedSize = 0;
193     for (TrieNode<V> child : element.getChilds()) {
194         countedSize += size(child);
195     }
196     countedSize += element.getValues().size();
197     return countedSize;
198 }
199
200 /**
201  * Print the tree.
202  */
203 public void print() {
204     print(0, root);
205 }
206
207 /**
208  *
209  * @param depth
210  *         Depth in which the node is.
211  * @param node
212  *         Node to print.
213  */
214 private void print(int depth, TrieNode<V> node) {
215     for (int i = 0; i < depth; i++) {
216         System.out.print("\t");
217     }
218     System.out.print(node.getKeySubstr() + ": ");
219     String values = node.getValues().stream().map(o -> o.toString())
220         .collect(Collectors.joining(", "));
221     System.out.println(values);
222     node.getChilds().forEach(trieNode -> print(depth + 1, trieNode));
223 }
224

```

```

1  /*
2   * OST - Uebungen 'Algorithmen & Datenstrukturen (AlgDat)'
3   * Version: Sun Oct 27 13:08:15 CET 2024
4   */
5
6  package ex07.solution.task03;
7
8  import java.util.Arrays;
9  import java.util.LinkedHashSet;
10 import java.util.Set;
11
12
13 public class BoyerMoore {
14
15     /**
16      * Total count of comparisons.
17      */
18     private static int totCount = 0;
19     /**
20      * Program-arguments (see Session-Log).
21      */
22     private static int task = 0;
23
24     public static int bmMatch(String t, int startIndex, int[] last, String p) {
25         int count = 0;
26         int n = t.length();
27         int m = p.length();
28         int i = startIndex + m - 1;
29         int j = m - 1;
30         if (i > n - 1)
31             return -1;
32         do {
33             count++;
34             if (task == 2) {
35                 System.out.format("i=%2d  j=%1d    chars i=%c  j=%c\n", i, j, t.charAt(i), p.c
36 harAt(j));
37             }
38             if (t.charAt(i) == p.charAt(j)) {
39                 if (j == 0) {
40                     System.out.println("Number of comparison: " + count);
41                     totCount += count;
42                     return i;
43                 } else {
44                     i--;
45                     j--;
46                 }
47             } else {
48                 i = i + m - Math.min(j, 1 + last[t.charAt(i)]);
49                 j = m - 1;
50             }
51         } while (i < n);
52
53         System.out.println("Number of comparison: " + count);
54         totCount += count;
55         return -1;
56     }
57
58     static int[] buildLastFunction(String p) {
59         int[] last = new int[128]; // for all ASCII-chars
60         for (int i = 0; i < last.length; i++) {
61             last[i] = -1;
62         }
63         for (int i = 0; i < p.length(); i++) {
64             last[p.charAt(i)] = i;
65         }
66         return last;
67     }
68 }

```

```

67 static void printLastFunction(String t, int[] last) {
68     char[] charArr = t.toCharArray();
69     Arrays.sort(charArr);
70     Set<Character> set = new LinkedHashSet<>();
71     for (int i = 0; i < charArr.length; i++) {
72         set.add(charArr[i]);
73     }
74     System.out.print("last: ");
75     for (char c: set) {
76         System.out.print(c+" ");
77     }
78     System.out.print("\n");
79     for (char c: set) {
80         System.out.format("%3d", last[c]);
81     }
82     System.out.println('\n');
83 }
84
85
86 public static void main(String[] args) {
87     String text;
88     String pattern;
89     pattern = text = "";
90     if (args.length == 0 || ((args.length == 1) && args[0].equals("1"))) {
91         text = "Anna Kurnikowa war eine Tennisspielerin. Sie spielte wieder ein wenig na
92         chdem ihre Beinverletzung fast wieder geheilt war.";
93         pattern = "ein";
94         task = 1;
95     } else if ((args.length == 1) && args[0].equals("2")) {
96         text = "adbaacaabedacedbccede";
97         pattern = "daeda";
98         task = 2;
99     } else {
100         if (args.length != 2) {
101             System.err.println("Bad number of arguments: " + args.length + " (expected: 2)
102             !");
103             System.exit(2);
104         }
105         text = args[0];
106         pattern = args[1];
107     }
108     System.out.println("Text      : "+text);
109     System.out.println("Pattern : "+pattern);
110     int res = 0;
111     int pos = 0;
112     int[] last = buildLastFunction(pattern);
113     printLastFunction(text, last);
114
115     while (res >= 0) {
116         //System.out.println(text.substring(pos));
117         res = bmMatch(text, pos, last, pattern);
118         if (res >= 0) {
119             System.out.println("Position: " + res);
120             pos = res + 1;
121             if (pos > text.length() - pattern.length()) {
122                 break;
123             }
124         }
125     }
126     System.out.println();
127     System.out.println("Total of comparison: " + totCount);
128 }

```

27.10.2024 13:08:15

KnuthMorrisPratt.java

Page 1/3

```

1  /*
2   * OST - Uebungen 'Algorithmen & Datenstrukturen (AlgDat)'
3   * Version: Sun Oct 27 13:08:15 CET 2024
4   */
5
6  package ex07.solution.task03;
7
8
9  public class KnuthMorrisPratt {
10
11     /**
12      * Total count of comparisons.
13      */
14     private static int totCount = 0;
15     /**
16      * Program-arguments (see Session-Log).
17      */
18     private static int task = 0;
19
20     public static int kmpMatch(String t, int startIndex, int[] fail, String p) {
21         int count = 0;
22         int n = t.length();
23         int m = p.length();
24         int i = startIndex;
25         int j = 0;
26         while (i < n) {
27             count++;
28             if (task == 2) {
29                 System.out.format("i=%2d  j=%1d    chars i=%c  j=%c\n", i, j, t.charAt(i), p.c
30 harAt(j));
31             }
32             if (t.charAt(i) == p.charAt(j)) {
33                 if (j == m - 1) {
34                     System.out.println("Number of comparison: " + count);
35                     totCount += count;
36                     return i - m + 1;
37                 } else {
38                     i++;
39                     j++;
40                 }
41             } else {
42                 if (j > 0) {
43                     j = fail[j - 1];
44                 } else {
45                     i++;
46                 }
47             }
48         }
49         System.out.println("Number of comparison: " + count);
50         totCount += count;
51         return -1;
52     }

```

```

53
54 static int[] buildFailureFunction(String p) {
55     int[] fail = new int[p.length()];
56     fail[0] = 0;
57     int i = 1;
58     int j = 0;
59     while (i < p.length()) {
60         if (p.charAt(i) == p.charAt(j)) {
61             fail[i] = j + 1;
62             i++;
63             j++;
64         } else if (j > 0) {
65             j = fail[j - 1];
66         } else {
67             fail[i] = 0;
68             i++;
69         }
70     }
71     return fail;
72 }
73
74 static void printFailureFunction(String p, int[] fail) {
75     System.out.print("fail: ");
76     for (int i = 0; i < p.length(); i++) {
77         System.out.print(p.charAt(i) + " ");
78     }
79     System.out.print("\n");
80     for (int i = 0; i < p.length(); i++) {
81         System.out.format("%3d", fail[i]);
82     }
83     System.out.println('\n');
84 }
85
86 public static void main(String[] args) {
87     String text;
88     String pattern;
89     pattern = text = "";
90     if (args.length == 0 || ((args.length == 1) && args[0].equals("1"))) {
91         text = "Anna Kurnikowa war eine Tennisspielerin. Sie spielte wieder ein wenig na
chdem ihre Beinverletzung fast wieder geheilt war.";
92         pattern = "ein";
93         task = 1;
94     } else if ((args.length == 1) && args[0].equals("2")) {
95         text = "dcdadaeddaedadaeddaed";
96         pattern = "daeda";
97         task = 2;
98     } else {
99         if (args.length != 2) {
100             System.err.println("Bad number of arguments: " + args.length + " (expected: 2)
!");
101             System.exit(2);
102         }
103         text = args[0];
104         pattern = args[1];
105     }
106     System.out.println("Text      : "+text);
107     System.out.println("Pattern  : "+pattern);
108     int res = 0;
109     int pos = 0;
110     int[] fail = buildFailureFunction(pattern);
111     printFailureFunction(pattern, fail);
112
113     while (res >= 0) {
114         //System.out.println(text.substring(pos));
115         res = kmpMatch(text, pos, fail, pattern);
116         if (res >= 0) {
117             pos = res + 1;
118             System.out.println("Position: " + res);
119         }
120     }

```

```
1  /*
2   * OST - Uebungen 'Algorithmen & Datenstrukturen (AlgDat)'
3   * Version: Mon Nov 11 14:49:06 CET 2024
4   */
5
6  package ex09.solution.task02;
7
8  import java.util.Deque;
9  import java.util.LinkedList;
10 import java.util.List;
11 import java.util.Set;
12 import java.util.TreeSet;
13 import java.util.stream.Collectors;
14
15 public class LCS {
16
17     private int tableL[][];
18     private String xStr;
19     private String yStr;
20
21     /**
22      * Calculates the L-Table.
23      * @param xStr First string (vertical axis of L-Table).
24      * @param yStr Second string (horizontal axis of L-Table).
25      * @return The calculated L-Table.
26      */
27     @SuppressWarnings("hiding")
28     public int[][] calculateTable(final String xStr, final String yStr) {
29         this.xStr = xStr;
30         this.yStr = yStr;
31         int n = xStr.length();
32         int m = yStr.length();
33         tableL = new int[n + 1][m + 1];
34         for (int i = 1; i <= n; i++) {
35             for (int j = 1; j <= m; j++) {
36                 if (xStr.charAt(i - 1) == yStr.charAt(j - 1)) {
37                     tableL[i][j] = tableL[i - 1][j - 1] + 1;
38                 } else {
39                     tableL[i][j] = Math.max(tableL[i - 1][j], tableL[i][j - 1]);
40                 }
41             }
42         }
43         return tableL;
44     }
45 }
```

```

45  /**
46   * Prints the L-Table (see 'Session-Log').
47   */
48
49  public void printTable() {
50      final int CELL_LEN = 4;
51      final int X_LEN = tableL.length;
52      final int Y_LEN = tableL[0].length;
53      System.out.println("LCS-Table:");
54      for (int y = 0; y < 2 * CELL_LEN; y++) {
55          System.out.print(' ');
56      }
57      for (int y = 0; y < Y_LEN - 1; y++) {
58          System.out.format("%4s", yStr.charAt(y));
59      }
60      System.out.println("");
61      for (int x = 0; x < X_LEN; x++) {
62          if (x == 0) {
63              for (int y = 0; y < CELL_LEN; y++) {
64                  System.out.print(' ');
65              }
66          } else {
67              System.out.format("%4s", xStr.charAt(x - 1));
68          }
69          for (int y = 0; y < Y_LEN; y++) {
70              System.out.format("%4d", tableL[x][y]);
71          }
72          System.out.println("");
73      }
74  }
75
76  enum Direction { XY, X, Y, END }
77
78  class CurrentPosition {
79      int x;
80      int y;
81      Direction currentDirection;
82
83      CurrentPosition(int x, int y) {
84          this.x = x;
85          this.y = y;
86          currentDirection = Direction.XY;
87      }
88
89      void nextDirection() {
90          for (Direction dir : Direction.values()) {
91              if (dir.ordinal() == currentDirection.ordinal() + 1) {
92                  currentDirection = dir;
93                  break;
94              }
95          }
96      }
97
98      Direction lastDirection() {
99          for (Direction dir : Direction.values()) {
100              if (dir.ordinal() == currentDirection.ordinal() - 1) {
101                  return dir;
102              }
103          }
104          return null;
105      }
106
107      int nextX() {
108          return x - (currentDirection == Direction.Y ? 0 : 1);
109      }
110
111      int nextY() {
112          return y - (currentDirection == Direction.X ? 0 : 1);
113      }
114  }
115

```


19.11.2024 12:52:13

AdjacencyListGraph.java

Page 2/6

```

71
72 @Override
73 public Vertex<V> opposite(Vertex<V> v, Edge<E> e)
74     throws IllegalArgumentException {
75     validate(v);
76     validate(e);
77     Vertex<V>[] vertices = endVertices(e);
78     if (v == vertices[0]) {
79         return vertices[1];
80     } else if (v == vertices[1]) {
81         return vertices[0];
82     } else {
83         throw new IllegalArgumentException("Edge is not incident to vertex");
84     }
85 }
86
87 @Override
88 public boolean areAdjacent(Vertex<V> v, Vertex<V> w)
89     throws IllegalArgumentException {
90     VertexImpl<V> vi = validate(v);
91     VertexImpl<V> wi = validate(w);
92     List<EdgeImpl<E>> incV = vi.getIncidentEdges();
93     List<EdgeImpl<E>> incW = wi.getIncidentEdges();
94     List<EdgeImpl<E>> inc;
95     if (incV.size() < incW.size()) {
96         inc = incV;
97     } else {
98         inc = incW;
99     }
100     for (EdgeImpl<E> e: inc) {
101         Vertex<V>[] endVertices = endVertices(e);
102         if (((endVertices[0] == v) && (endVertices[1] == w)) ||
103             ((endVertices[0] == w) && (endVertices[1] == v))) {
104             return true;
105         }
106     }
107     return false;
108 }
109
110 @Override
111 public Vertex<V> insertVertex(V element) {
112     VertexImpl<V> v = new VertexImpl<>(element);
113     Node<VertexImpl<V>> node;
114     node = vertices.insert(v);
115     v.setBackReference(node);
116     return v;
117 }
118
119 @Override
120 public Edge<E> insertEdge(Vertex<V> v, Vertex<V> w, E element)
121     throws IllegalArgumentException {
122     VertexImpl<V> vi = validate(v);
123     VertexImpl<V> wi = validate(w);
124     EdgeImpl<E> e = new EdgeImpl<>(v, w, element);
125     Node<EdgeImpl<E>> node;
126     node = edges.insert(e);
127     e.setBackReference(node);
128     node = vi.incidentList.insert(e);
129     e.backReferences[0] = node;
130     node = wi.incidentList.insert(e);
131     e.backReferences[1] = node;
132     return e;
133 }

```

```

134
135 @Override
136 @SuppressWarnings("unchecked")
137 public V removeVertex(Vertex<V> v) throws IllegalArgumentException {
138     VertexImpl<V> vi = validate(v);
139     List<EdgeImpl<E>> edges = vi.getIncidentEdges();
140     for (EdgeImpl<E> e : edges) {
141         removeEdge(e);
142     }
143     vertices.remove((Node<VertexImpl<V>>) vi.getBackReference());
144     vi.setBackReference(null); // mark vertex as <defunct>
145     return v.getElement();
146 }
147
148 @Override
149 @SuppressWarnings("unchecked")
150 public E removeEdge(Edge<E> e) throws IllegalArgumentException {
151     EdgeImpl<E> ei = validate(e);
152     Vertex<V>[] vertices = ei.getEndpoints();
153     Node<?>[] backRefs = ei.getBackReferences();
154     VertexImpl<V> v0 = (VertexImpl<V>) vertices[0];
155     v0.removeIncident(backRefs[0]);
156     VertexImpl<V> v1 = (VertexImpl<V>) vertices[1];
157     v1.removeIncident(backRefs[1]);
158     edges.remove((Node<EdgeImpl<E>>) ei.getBackReference());
159     ei.setBackReference(null); // mark edge as <defunct>
160     return e.getElement();
161 }
162
163 protected VertexImpl<V> validate(Vertex<V> v)
164     throws IllegalArgumentException {
165     VertexImpl<V> vi = (VertexImpl<V>) v;
166     if (!vi.validate(this))
167         throw new IllegalArgumentException("Invalid vertex");
168     return vi;
169 }
170
171 protected EdgeImpl<E> validate(Edge<E> e) throws IllegalArgumentException {
172     EdgeImpl<E> ei = (EdgeImpl<E>) e;
173     if (!ei.validate(this))
174         throw new IllegalArgumentException("Invalid edge");
175     return ei;
176 }
177
178 @Override
179 public String toString() {
180     StringBuilder sb = new StringBuilder();
181     for (VertexImpl<V> v : vertices.getItems()) {
182         sb.append(v.getElement() + " -> ");
183         for (EdgeImpl<E> e : v.getIncidentEdges()) {
184             Vertex<V> w = opposite(v, e);
185             sb.append("(" + w + ", " + e + " ");
186         }
187         sb.append("\n");
188     }
189     return sb.toString();
190 }
191
192 public void print() {
193     System.out.println("Graph:");
194     System.out.println(toString());
195 }

```

```

196
197 /**
198  * Prints the content of the vertex- and edge-sequences.
199  *
200  * listID:NodeNr|Item(BackRef)|prev<>next [: endpoints[2] : backrefs[2]]
201  * listID: "v-seq" | "e-seq"
202  * NodeNr: `listID`:"head"|`nr`|"tail"
203  * Item: Node.item.toString()
204  * BackRef: Back-Reference of Item
205  * prev: Node.prev
206  * next: Node.next
207  */
208 protected void printDiagnostic() {
209     Map<Integer, String> objIdMap = new LinkedHashMap<>();
210     fillUp(objIdMap);
211     System.out.println("v-seq:");
212     vertices.printDiagnostic("    ", objIdMap);
213     System.out.println("e-seq:");
214     edges.printDiagnostic("    ", objIdMap);
215 }

```

```

216  /**
217  * Traverses all lists and puts each object into the map (once!).
218  *
219  * @param objIdMap
220  *       Mapping of objects to its String-ID.
221  * @throws IllegalStateException
222  *       When an object is inserted more than once into the map.
223  */
224  @SuppressWarnings("rawtypes")
225  protected void fillUp(Map<Integer, String> objIdMap)
226      throws IllegalStateException {
227      String listID = "v-seq";
228      Node head = vertices.getHead();
229      Node tail = vertices.getTail();
230      putInMap(head, listID + ":head", objIdMap);
231      int i = 0;
232      Node cursor = head.getNext();
233      while (cursor != tail) {
234          putInMap(cursor, listID + ":" + i, objIdMap);
235          putInMap(cursor.getItem(), cursor.getItem().getElement().toString(), objIdMap);
236          String incListID = "inc-seq";
237          VertexImpl v = (VertexImpl) cursor.getItem();
238          Node incHead = v.incidentList.getHead();
239          Node incTail = v.incidentList.getTail();
240          putInMap(incHead, incListID + ":" + i + "-head", objIdMap);
241          int j = 0;
242          Node incCursor = incHead.getNext();
243          while (incCursor != incTail) {
244              putInMap(incCursor, incListID + ":" + i + "-" + j, objIdMap);
245              incCursor = incCursor.getNext();
246              j++;
247          }
248          putInMap(incTail, incListID + ":" + i + "-tail", objIdMap);
249          cursor = cursor.getNext();
250          i++;
251      }
252      putInMap(tail, listID + ":tail", objIdMap);
253
254      listID = "e-seq";
255      head = edges.getHead();
256      tail = edges.getTail();
257      putInMap(head, listID + ":head", objIdMap);
258      i = 0;
259      cursor = head.getNext();
260      while (cursor != tail) {
261          putInMap(cursor, listID + ":" + i, objIdMap);
262          cursor = cursor.getNext();
263          i++;
264      }
265      putInMap(tail, listID + ":tail", objIdMap);
266  }
267
268  private static void putInMap(Object obj, String objId, Map<Integer, String> objIdMap
269  )
270      throws IllegalStateException {
271      if (objIdMap.get(obj.hashCode()) == null) {
272          objIdMap.put(obj.hashCode(), objId);
273      } else {
274          throw new IllegalStateException(objId + ": exists already!");
275      }
276  }

```

```

277 class VertexImpl<V extends Comparable<V>> extends BackRefNodeItem<V>
278     implements Vertex<V> {
279
280     private BackRefList<EdgeImpl<E>> incidentList;
281
282     protected VertexImpl(V element) {
283         setElement(element);
284         incidentList = new BackRefList<>();
285     }
286
287     @Override
288     public V getElement() {
289         return super.getElement();
290     }
291
292     protected List<EdgeImpl<E>> getIncidentEdges() {
293         return incidentList.getItems();
294     }
295
296     @SuppressWarnings({ "rawtypes", "unchecked" })
297     protected void removeIncident(Node<?> backRef) {
298         incidentList.remove((Node) backRef);
299     }
300
301     protected BackRefList<EdgeImpl<E>> getIncidentList() {
302         return incidentList;
303     }
304
305     protected boolean validate(Graph<V, E> graph) {
306         return (AdjacencyListGraph.this == graph) && (getBackReference() != null);
307     }
308 }
309
310
311 class EdgeImpl<E extends Comparable<E>> extends BackRefNodeItem<E>
312     implements Edge<E> {
313
314     private Vertex<V>[] endpoints;
315     private Node<?>[] backReferences;
316
317     @SuppressWarnings("unchecked")
318     protected EdgeImpl(Vertex<V> v, Vertex<V> w, E element) {
319         setElement(element);
320         endpoints = new Vertex[] { v, w };
321         backReferences = new Node[2];
322     }
323
324     @Override
325     public E getElement() {
326         return super.getElement();
327     }
328
329     protected Vertex<V>[] getEndpoints() {
330         return endpoints;
331     }
332
333     protected Node<?>[] getBackReferences() {
334         return backReferences;
335     }
336
337     protected boolean validate(Graph<V, E> graph) {
338         return (AdjacencyListGraph.this == graph) && (getBackReference() != null);
339     }
340 }
341
342 }
343
344 }

```

```

1  /*
2   * OST - Uebungen 'Algorithmen & Datenstrukturen (AlgDat)'
3   * Version: Tue Nov 26 17:45:51 CET 2024
4   */
5
6  package ex11.solution.task02;
7
8  import java.util.Map;
9
10 import ex11.solution.task02.graph.AdjacencyListGraphEnE;
11 import ex11.solution.task02.graph.Edge;
12 import ex11.solution.task02.graph.GraphEnE;
13 import ex11.solution.task02.graph.Vertex;
14
15 public class DepthFirstSearch<V extends Comparable<V>> {
16
17     public enum VertexLabelDFS {
18         UNEXPLORED, VISITED
19     }
20
21     public enum EdgeLabelDFS {
22         UNEXPLORED, DISCOVERY, BACK
23     }
24
25     protected Map<Vertex<V>, VertexLabelDFS> vertexMap;
26     protected Map<Edge, EdgeLabelDFS> edgeMap;
27
28     public void search(GraphEnE<V> graph) {
29         vertexMap = ((AdjacencyListGraphEnE<V>)graph).getDFSVertexMap();
30         edgeMap = ((AdjacencyListGraphEnE<V>)graph).getDFSEdgeMap();
31
32         for (Vertex<V> u : graph.vertices()) {
33             vertexMap.put(u, VertexLabelDFS.UNEXPLORED);
34         }
35         for (Edge e : graph.edges()) {
36             edgeMap.put(e, EdgeLabelDFS.UNEXPLORED);
37         }
38         for (Vertex<V> v : graph.vertices()) {
39             if (vertexMap.get(v) == VertexLabelDFS.UNEXPLORED) {
40                 search(graph, v);
41             }
42         }
43     }
44
45     protected void search(GraphEnE<V> graph, Vertex<V> v) {
46         System.out.println("DepthFirstSearch.search(): v = " + v);
47         vertexMap.put(v, VertexLabelDFS.VISITED);
48         for (Edge e : graph.incidentEdges(v)) {
49             System.out.println("  e = " + e);
50             if (edgeMap.get(e) == EdgeLabelDFS.UNEXPLORED) {
51                 Vertex<V> w = graph.opposite(v, e);
52                 System.out.println("    w = " + w);
53                 if (vertexMap.get(w) == VertexLabelDFS.UNEXPLORED) {
54                     edgeMap.put(e, EdgeLabelDFS.DISCOVERY);
55                     search(graph, w);
56                 } else {
57                     edgeMap.put(e, EdgeLabelDFS.BACK);
58                 }
59             }
60         }
61     }
62
63     public void print() {
64         System.out.println("\nDepthFirstSearch.print():");
65         System.out.println("Vertex-Map : " + vertexMap);
66         System.out.println("Edge-Map   : " + edgeMap);
67     }
68
69 }
70

```

```
1  /*
2  * OST - Uebungen 'Algorithmen & Datenstrukturen (AlgDat)'
3  * Version: Tue Nov 26 17:45:51 CET 2024
4  */
5
6  package ex11.solution.task02;
7
8  import java.util.ArrayList;
9  import java.util.LinkedList;
10 import java.util.Map;
11
12 import ex11.solution.task02.graph.AdjacencyListGraphEnE;
13 import ex11.solution.task02.graph.Edge;
14 import ex11.solution.task02.graph.GraphEnE;
15 import ex11.solution.task02.graph.Vertex;
16
17 public class BreadthFirstSearch<V> extends Comparable<V>> {
18
19     public enum VertexLabelBFS {
20         UNEXPLORED, VISITED
21     }
22
23     public enum EdgeLabelBFS {
24         UNEXPLORED, DISCOVERY, CROSS
25     }
26
27     protected Map<Vertex<V>, VertexLabelBFS> vertexMap;
28     protected Map<Edge, EdgeLabelBFS> edgeMap;
29
30     public void search(GraphEnE<V> graph) {
31         vertexMap = ((AdjacencyListGraphEnE<V>)graph).getBFSVertexMap();
32         edgeMap = ((AdjacencyListGraphEnE<V>)graph).getBFSEdgeMap();
33
34         for (Vertex<V> u : graph.vertices()) {
35             vertexMap.put(u, VertexLabelBFS.UNEXPLORED);
36         }
37         for (Edge e : graph.edges()) {
38             edgeMap.put(e, EdgeLabelBFS.UNEXPLORED);
39         }
40         for (Vertex<V> v : graph.vertices()) {
41             if (vertexMap.get(v) == VertexLabelBFS.UNEXPLORED) {
42                 search(graph, v);
43             }
44         }
45     }
46 }
```

26.11.2024 17:45:51

BreadthFirstSearch.java

Page 2/2

```

46
47 protected void search(GraphEnE<V> graph, Vertex<V> s) {
48     System.out.println("BreadhFirstSearch.search(): s = " + s);
49     ArrayList<LinkedList<Vertex<V>>> lists = new ArrayList<>();
50     LinkedList<Vertex<V>> listI = new LinkedList<>();
51     lists.add(listI);
52     listI.add(s);
53     vertexMap.put(s, VertexLabelBFS.VISITED);
54     int i = 0;
55     while (!listI.isEmpty()) {
56         LinkedList<Vertex<V>> listIplus1 = new LinkedList<>();
57         lists.add(listIplus1);
58         for (Vertex<V> v : listI) {
59             System.out.println("    v = " + v);
60             for (Edge e : graph.incidentEdges(v)) {
61                 System.out.println("        e = " + e);
62                 if (edgeMap.get(e) == EdgeLabelBFS.UNEXPLORED) {
63                     Vertex<V> w = graph.opposite(v, e);
64                     System.out.println("            w = " + w);
65                     if (vertexMap.get(w) == VertexLabelBFS.UNEXPLORED) {
66                         edgeMap.put(e, EdgeLabelBFS.DISCOVERY);
67                         System.out.println("                e = DISCOVERY");
68                         vertexMap.put(w, VertexLabelBFS.VISITED);
69                         listIplus1.add(w);
70                     } else {
71                         edgeMap.put(e, EdgeLabelBFS.CROSS);
72                         System.out.println("                e = CROSS");
73                     }
74                 }
75             }
76         }
77         i++;
78         listI = lists.get(i);
79     }
80 }
81
82 public void print() {
83     System.out.println("\nBreadthFirstSearch.print():");
84     System.out.println("Vertex-Map : " + vertexMap);
85     System.out.println("Edge-Map   : " + edgeMap);
86 }
87
88 }
89

```



```

1  /*
2   * OST - Uebungen 'Algorithmen & Datenstrukturen (AlgDat)'
3   * Version: Fri Nov 29 19:05:45 CET 2024
4   */
5
6  package ex12.solution.task02;
7
8  import java.util.LinkedHashMap;
9  import java.util.Map;
10
11 import ex12.solution.task02.graph.AdjacencyListDigraph;
12 import ex12.solution.task02.graph.Digraph;
13 import ex12.solution.task02.graph.Edge;
14 import ex12.solution.task02.graph.Vertex;
15
16 public class DirectedDFS<V> extends Comparable<V>> {
17
18     public enum VertexLabel {
19         UNEXPLORED, VISITED
20     }
21
22     public enum EdgeLabel {
23         UNEXPLORED, DISCOVERY, BACK, FORWARD, CROSS
24     }
25
26     protected Map<Vertex<V>, VertexLabel> vertexMap;
27     protected Map<Edge, EdgeLabel> edgeMap;
28     /** The parent-map maps a child to its parent. */
29     protected Map<Vertex<V>, Vertex<V>> parentMap = new LinkedHashMap<>();
30     /** The indentation of the colon(:) while tracing (see Session-Log). */
31     protected final static int INDENT = 21;
32
33
34     public void search(Digraph<V> graph) {
35         vertexMap = ((AdjacencyListDigraph<V>)graph).getVertexMap();
36         edgeMap = ((AdjacencyListDigraph<V>)graph).getEdgeMap();
37
38         for (Vertex<V> u : graph.vertices()) {
39             vertexMap.put(u, VertexLabel.UNEXPLORED);
40         }
41         for (Edge e : graph.edges()) {
42             edgeMap.put(e, EdgeLabel.UNEXPLORED);
43         }
44         for (Vertex<V> v : graph.vertices()) {
45             if (vertexMap.get(v) == VertexLabel.UNEXPLORED) {
46                 search(graph, v);
47             }
48         }
49     }
50
51     protected void search(Digraph<V> graph, Vertex<V> v) {
52         System.out.format("%-" + INDENT + "s: %s\n", "DirectedDFS.search()", v);
53         vertexMap.put(v, VertexLabel.VISITED);
54         for (Edge e : graph.outgoingEdges(v)) {
55             System.out.format("%-" + INDENT + "s: %s: ", "Testing", e);
56             if (edgeMap.get(e) == EdgeLabel.UNEXPLORED) {
57                 Vertex<V> w = graph.opposite(v, e);
58                 if (vertexMap.get(w) == VertexLabel.UNEXPLORED) {
59                     edgeMap.put(e, EdgeLabel.DISCOVERY);
60                     parentMap.put(w, v);
61                     System.out.println(EdgeLabel.DISCOVERY);
62                     search(graph, w);
63                 } else {
64                     EdgeLabel edgeLabel = identifyKindOfEdge(v, w);
65                     edgeMap.put(e, edgeLabel);
66                     System.out.println(edgeLabel);
67                 }
68             }
69         }
70     }

```

```
71
72     protected EdgeLabel identifyKindOfEdge(Vertex<V> current, Vertex<V> opposite) {
73         if (isAncestor(opposite, current)) {
74             return EdgeLabel.BACK;
75         }
76         else if (isAncestor(current, opposite)) {
77             return EdgeLabel.FORWARD;
78         }
79         else {
80             return EdgeLabel.CROSS;
81         }
82     }
83
84     /** Is upperVertex an ancestor of lowerVertex? */
85     protected boolean isAncestor(Vertex<V> upperVertex, Vertex<V> lowerVertex) {
86         for (Vertex<V> v = lowerVertex; v != null; v = parentMap.get(v)) {
87             if (v == upperVertex) {
88                 return true;
89             }
90         }
91         return false;
92     }
93
94     public void print() {
95         System.out.println("\nDirectedDFS.print():");
96         System.out.println("Vertex-Map : " + vertexMap);
97         System.out.println("Edge-Map   : " + edgeMap);
98     }
99
100 }
101
102
```

```
1  /*
2   * OST - Uebungen 'Algorithmen & Datenstrukturen (AlgDat)'
3   * Version: Mon Dec 9 14:21:59 CET 2024
4   */
5
6  package ex13.solution.task02;
7
8  import java.util.LinkedHashMap;
9  import java.util.Map;
10
11  import ex13.solution.task02.graph.AdjacencyListGraph;
12  import ex13.solution.task02.graph.Edge;
13  import ex13.solution.task02.graph.Graph;
14  import ex13.solution.task02.graph.Vertex;
15
16  public class Dijkstra<V> extends Comparable<V>> {
17
18      public Map<Vertex<V>, Integer> distances = getDistanceMap();
19      private Map<Vertex<V>, Entry<Integer, Vertex<V>>> locators = getLocatorMap();
20      private Map<Vertex<V>, Edge<Integer>> parents = getParentMap();
21      private AdaptablePriorityQueue<Integer, Vertex<V>> apq;
22      private Graph<V, Integer> theGraph;
23
24      public void distances(Graph<V, Integer> graph, Vertex<V> s)
25          throws FullPriorityQueueException, InvalidEntryException {
26          theGraph = graph;
27          apq = getAdaptablePriorityQueue(graph);
28
29          for (Vertex<V> v : graph.vertices()) {
30              if (v == s) {
31                  distances.put(v, 0);
32              } else {
33                  distances.put(v, Integer.MAX_VALUE);
34              }
35              Entry<Integer, Vertex<V>> l = apq.insert(distances.get(v), v);
36              locators.put(v, l);
37              parents.put(v, null);
38          }
39          while (!apq.isEmpty()) {
40              Vertex<V> u = apq.removeMin().getValue();
41              for (Edge<Integer> e : graph.incidentEdges(u)) {
42                  Vertex<V> z = graph.opposite(u, e);
43                  int r = distances.get(u) + e.getElement();
44                  if (r < distances.get(z)) {
45                      distances.put(z, r);
46                      parents.put(z, e);
47                      apq.replaceKey(locators.get(z), r);
48                  }
49              }
50          }
51      }
```

9.12.2024 14:21:59

Dijkstra.java

Page 2/2

```
52
53 public Graph<V, Integer> getGraph() {
54     return new AdjacencyListGraph<>();
55 }
56
57 protected AdaptablePriorityQueue<Integer, Vertex<V>> getAdaptablePriorityQueue(
58     Graph<V, Integer> graph) {
59     return new AdaptablePriorityQueue<>(graph.numVertices());
60 }
61
62 protected Map<Vertex<V>, Integer> getDistanceMap() {
63     return new LinkedHashMap<>();
64 }
65
66 protected Map<Vertex<V>, Entry<Integer, Vertex<V>>> getLocatorMap() {
67     return new LinkedHashMap<>();
68 }
69
70 protected Map<Vertex<V>, Edge<Integer>> getParentMap() {
71     return new LinkedHashMap<>();
72 }
73
74 public void print() {
75     System.out.println("\nDistances:");
76     for (Vertex<V> v : theGraph.vertices()) {
77         System.out.println(v + ": " + distances.get(v));
78     }
79 }
80
81 }
```